get_status

Command: `get_status`

Description

Overview

The `get_status` command retrieves the current status of a Zebra handheld RFID reader. The response includes power source information, radio activity and connection state, device temperature, system time, Network Time Protocol (NTP) synchronization details, terminal connection status, and battery health metrics.

Use this command to monitor device health, verify connectivity before starting operations, and collect telemetry data for fleet management dashboards. The command requires no configuration parameters beyond the standard envelope fields.

> **Note** • The `deviceStatus` object in the response is optional. When the device cannot retrieve status information (response code `3`), the response omits this object.

Command details:

Pattern Name	Device Status Retrieval
Communication Type	Bidirectional (Cloud to Device, Device to Cloud)
Applies To	RFD40 Series, RFD90 Series

Response fields:

Field	What it tells you
`powerSource`	How the scanner is powered — DC, Wall Charger, USB, or Cradle
`radioActivity`	Whether the RFID radio is currently active or idle
`radioConnection`	Whether the radio module is connected or disconnected
`hostname`	The scanner's network hostname
`systemTime`	Current device clock in ISO 8601 format
`temperature`	Internal temperature in °C — monitor for overheating
`ntp`	Clock sync status — offset (ms drift) and reach (sync success history)

Field	What it tells you
`terminalConnection`	Paired phone/tablet — connection status and type (Bluetooth, USB, CIO)
`batteryStatus`	Battery capacity (mAh), charge %, health (Good/Average/Poor), charge state

Use Cases

Fleet Health Monitoring

A warehouse management platform polls all readers at five-minute intervals using `get_status`. The system collects `batteryStatus.chargePercentage`, `temperature`, and `radioConnection` values from each device. When `chargePercentage` drops below 20 or `temperature` exceeds 45 degrees Celsius, the platform generates an alert and schedules the device for charging or cooldown.

Pre-Operation Readiness Check

Before starting an inventory scan, a mobile application sends `get_status` to verify that `radioConnection` is `CONNECTED` and `batteryStatus.stateOfHealth` is not `POOR`. If either condition fails, the application prompts the operator to reconnect the sled or replace the battery before proceeding with `control_operation`.

Battery Replacement Planning

A device management system tracks `batteryStatus.stateOfHealth` across the fleet over time. When a device transitions from `GOOD` to `AVERAGE`, the system adds the device to a scheduled replacement queue. Devices reporting `POOR` state of health are flagged for immediate battery swap.

Time Synchronization Verification

An enterprise system queries `get_status` to check the `ntp.offset` and `ntp.reach` values. A high offset indicates clock drift that can cause timestamp mismatches in tag event data. The system triggers an NTP resynchronization when the offset exceeds the configured threshold.

Terminal Connection Diagnostics

A field service technician troubleshoots connectivity issues between an RFD40 sled and a host mobile computer. The technician sends `get_status` and checks `terminalConnection.status` and `terminalConnection.type` to confirm whether the sled is connected over Bluetooth, USB, or CIO, and whether the connection is active.

Request

Request Fields

Field	Type	Required	Description
`command`	string (enum)	**Yes**	Must be `get_status`. Tells the scanner to return its current health and status.

Field	Type	Required	Description
`requestId`	string	**Yes**	A unique ID you assign to this request. The scanner echoes it back in the response so you can match requests to responses.

Request Examples

Generated Example

```
{
  "command": "get_status",
  "requestId": "abcd123"
}
```

Response

Response Fields

Field	Type	Required	Description
`command`	string	**Yes**	Echoes back the command name — always `get_status` for this response.
`requestId`	string	**Yes**	Echoes back the requestId you sent, so you can match this response to your original request.
`apiVersion`	string (enum)	**Yes**	The API version of the IoT Connector responding to this command.

Field	Type	Required	Description
`deviceStatus`	object	No	Real-time device health — power source, RFID radio state, temperature, battery level and health, NTP clock sync, and paired terminal (phone/tablet) connection.
`deviceStatus.powerSource`	string (enum)	**Yes**	The source of power for the device (e.g., DC, Wall Charger, USB, or Cradle).
`deviceStatus.radioActivity`	string (enum)	**Yes**	The activity status of the device's radio (e.g., Active or Inactive).
`deviceStatus.radioConnection`	string (enum)	**Yes**	The connection status of the device's radio (e.g., Connected or Disconnected).
`deviceStatus.hostname`	string	**Yes**	The hostname of the device on the network.
`deviceStatus.systemTime`	string	**Yes**	The system time of the device in ISO 8601 format.
`deviceStatus.temperature`	integer	**Yes**	The current temperature of the device in degrees Celsius.
`deviceStatus.ntp`	object	No	Details about the Network Time Protocol (NTP) synchronization.
`deviceStatus.ntp.offset`	integer	No	The offset in milliseconds between the device's clock and the NTP server.
`deviceStatus.ntp.reach`	integer	No	The reachability score of the NTP server.

Field	Type	Required	Description
`deviceStatus.terminalConnection`	object	No	Details about the terminal connection to the device.
`deviceStatus.terminalConnection.status`	string (enum)	No	The connection status of the terminal (e.g., Connected or Disconnected).
`deviceStatus.terminalConnection.type`	string (enum)	No	The type of terminal connection (e.g., Bluetooth, CIO, or USB).
`deviceStatus.batteryStatus`	object	No	Information about the battery status of the device.
`deviceStatus.batteryStatus.capacity`	integer	No	The battery capacity remaining in mAh.
`deviceStatus.batteryStatus.stateOfHealth`	string (enum)	No	The overall health of the battery (e.g., Good, Average, or Poor).
`deviceStatus.batteryStatus.chargePercentage`	integer	No	The percentage of battery charge remaining.
`deviceStatus.batteryStatus.chargeStatus`	integer	No	The charging status of the device battery: 0 - Charger not connected, 1 - Charger connected and charging in progress, 2 - Charger connected and battery is 100%.
`response`	object	**Yes**	Status of the command execution — check `code` to confirm success (0) or identify the error.

Field	Type	Required	Description
`response.code`	integer	**Yes**	Response code indicating success or failure. • **0** — Success • **3** — Not able to retrieve information
`response.description`	string	**Yes**	Human-readable description of the response code (e.g. "Success", "Invalid payload").

Response Examples

Device status info

```
{
  "command": "get_status",
  "requestId": "abcd123",
  "apiVersion": "V1.1",
  "deviceStatus": {
    "powerSource": "USB",
    "radioActivity": "INACTIVE",
    "radioConnection": "CONNECTED",
    "systemTime": "2024-02-26T13:45:53.728Z",
    "temperature": 32,
    "ntp": {
      "offset": 0,
      "reach": 0
    },
    "terminalConnection": {
      "status": "CONNECTED",
      "type": "USB"
    },
    "batteryStatus": {
      "capacity": 6400,
      "stateOfHealth": "GOOD",
      "chargePercentage": 100,
      "chargeStatus": 1
    }
  },
  "response": {
    "code": 0,
    "description": "Success"
  }
}
```

Status / Error Codes

Code	Status Constant	Description
0	`IOT_STATUS_SUCCESS`	Success
3	`IOT_ERROR_INFO_NOT_AVAILABLE`	Not able to retrieve information